

The Alberta Buck - Publication and Deployment (v0.2)

What is published, why it is built the way it is, and what remains

Perry Kundert

2026-07-26 12:00:00

The BUCK codebase had one consumer: this checkout. Every entry point was a **make** target, every import path a `sys.path` shim, every binary artifact gitignored and rebuilt locally on demand.

The **Alberta Buck game** – a board projected on a TV, personal devices showing private net worth, real economic decisions played against real BUCK mechanics – is the forcing function that ends that. A game runs on someone else's machine.

Seven artifacts are now published and installable. Getting there was less about packaging than about **determinism**: an artifact you cannot reproduce is an artifact you cannot honestly publish, and the repository turned out to hold two independent sources of irreproducible binaries. Both are gone. This document records what is live, why the build is shaped the way it is, and what remains.

Two facts still govern everything. **Publishing is publishing**: a private GitHub repository and a public package are entirely compatible, but the tarball we upload contains the source, so the question is never "can we publish while private" – it is "which code are we ready to make public". And **the first upload is irreversible**: crates.io versions cannot be deleted, PyPI filenames cannot be reused, npm unpublish is a 72-hour window.

Contents

1	Status	3
2	The architecture, and why	3
2.1	Names: everything is <code>alberta-buck-*</code>	3
2.2	Licence: CAL-1.0 for the kernels, GPL-3.0 for contracts and sim	4
2.3	Versions: lockstep at major.minor, patch floats	5
2.4	One compiler for everything we own	6
2.5	Third-party artifacts come from their publishers	6
2.6	The contracts have no addresses	7
3	Mechanics worth not rediscovering	8
3.1	The first publish is always manual	8
3.2	Build outputs are not published by default	8
3.3	Dependencies must be pinned to be reproducible	8
4	Remaining decisions	9
4.1	D1: does the sim's source go public?	9
4.2	D2: where the game's authority lives	9
5	The way forward	10
5.1	P2.5b: <code>alberta-buck-contracts</code> on PyPI	10
5.2	P3: <code>alberta-buck-core</code> on PyPI, and the pure/binary split	10
5.3	P4: <code>alberta-buck-kernel</code> wheels	11
5.4	P5: <code>alberta-buck</code> on PyPI	11
5.5	P6: what the game adds	12
5.6	P7: making the repository public	13
6	Risks	13

1 Status

Published, and each verified by installing from the registry into an empty project outside this checkout:

artifact	registry	version	licence
alberta-buck-math	crates.io	0.1.0	CAL-1.0
alberta-buck-identity	crates.io	0.1.0	CAL-1.0
alberta-buck-registry	crates.io	0.1.0	CAL-1.0
alberta-buck-wallet	crates.io	0.1.0	CAL-1.0
alberta-buck-kernel	npm	0.1.1	CAL-1.0
alberta-buck-core	npm	0.1.1	CAL-1.0
alberta-buck-contracts	npm	0.1.0	GPL-3.0-or-later

Remaining:

#	artifact / step	registry	blocked on
2.5	alberta-buck-contracts	PyPI	a first-publish credential
3	alberta-buck-core	PyPI	the pure/binary split
4	alberta-buck-kernel	PyPI	the wheel matrix
5	alberta-buck	PyPI	D1 – is the sim public?
6	alberta-buck-game + zomes	npm	D2 – where authority lives
7	repository public	GitHub	a deliberate decision

In place: `release-crates.yml` and `release-npm.yml` (trusted publishing, no stored tokens), `LICENSE / core/LICENSE / LICENSING.md / NOTICE`, a Rust toolchain and `binaryen` in the flake, `Cargo.lock` and `foundry.lock` committed. **Nothing is pushed to GitHub yet**, so neither workflow has ever run – and the source of seven published artifacts exists on one machine.

2 The architecture, and why

Six decisions shape everything above. They are settled; the reasoning is recorded because the *consequences* keep surfacing, not to reopen them.

2.1 Names: everything is alberta-buck-*

Every published distribution carries the project prefix, on every registry. The same name on two registries names that language’s build of the same thing

– `alberta-buck-core` is the Python platform on PyPI and the JS platform on npm, deliberately.

Import names do **not** change: `import buck_core`, use `buck_math::`, `import { Session } from "alberta-buck-core"`. Distribution name and import name are different things (`python-slip39` imports as `slip39`). For the crates this needs an explicit `[lib] name`, since cargo would otherwise derive `alberta_buck_math` from the package name. Directory names do not change either: `core/rust/buck-math` stays.

The cost of unscoped names, paid twice: the first publish of a **new** package needs a broadly-scoped token, because a package that does not yet exist cannot appear in a token’s package picker. With `@alberta-buck/*` a token could have been scoped to the scope. The ergonomics of npm i `alberta-buck-core` were judged worth it; the `@alberta-buck` scope is reserved anyway, against typosquatting.

2.2 Licence: CAL-1.0 for the kernels, GPL-3.0 for contracts and sim

The rule: start restrictive and relax deliberately. A licence can be loosened later, or granted separately to a counterparty; it can never be tightened on a version already published.

The goal is that an organization intending to build a *liberty-restricting* system – custodial control of other people’s keys, freezable balances, users denied their own records – is at minimum operating out of licence. No OSI-approved licence can do this by restricting *purpose*: OSD 6 forbids discrimination against fields of endeavour.

The **Cryptographic Autonomy License** (CAL-1.0, OSI-approved 2020, drafted for Holochain) restricts *architecture* instead, which is both enforceable and self-executing:

section	obligation
4.1	provide Recipients the Source Code, or unrestricted network access to it
4.1.2	modifications go out under CAL-1.0 or a compatible licence
4.1.3	a 90-day embargo is permitted for coordinated vulnerability disclosure
4.2.1	give each Recipient a no-charge copy of their own User Data
4.2.2	no cryptographic or technological measure limiting a Recipient’s access to functionality
4.2.3	no contractual restriction on a Recipient exercising the same permissions independently
4.5	Combined Work Exception: per-file opt-in permitting combination into a larger work under its own licence

A system that holds users’ keys, can freeze their balances, or withholds

their records **cannot comply with 4.2 while using CAL-covered code**. It is out of licence the day it ships – no intent need be proven, and no field of endeavour is discriminated against.

Applied to the kernels, the bindings and both platform libraries, because that is where the leverage is: anyone building a control system needs the identity, wallet and Notes kernels. CAL anywhere else would be decorative. The contracts and `alberta_buck` are GPL-3.0-or-later, which all 49 `src/*.sol` files already declared.

Consequences to plan for:

- **The Combined Work Exception is the partner path, and it is one-way.** Marking a file CWE cannot be undone for a published version. Mark it narrowly and late – at the binding surface, if a real integrator needs it – never across the kernels wholesale. **No file carries it today.**
- **Dual licensing is the enforcement lever**, and works only while the copyright is held in one place. An organization that cannot comply must come and ask, and may be declined. This requires a **CLA** the moment the repository is public and outside contributions arrive.
- **Expect friction.** CAL is young and thinly litigated; counsel will read it rather than recognize it. That is the instrument working, and a real cost against adoption.
- **The licence governs code, not ideas.** The papers are public; the mechanism can be reimplemented. What CAL buys is that it cannot be done *with our code* while denying users their autonomy.

2.3 Versions: lockstep at major.minor, patch floats

The kernels are one artifact compiled three ways, so the bit-identical guarantee must be anchored to a version – but it is a claim about *semantics*, and only major.minor can change semantics.

- `alberta-buck-math 0.1.*` in Rust, JS and Python compute identically. The cross-language vector suites enforce this, and may assume it.
- A patch release in one language does **not** oblige the others. The JS packages went to 0.1.1 for a `wasm-opt` size pass that did not touch a line of Rust; dragging four identical crates along would have been ceremony, not information.

- A major or minor bump is a cross-language event. It lands everywhere, or the guarantee stops being true.

Inter-crate requirements stay caret ("0.1"), and `release-crates.yml` guards that they track the workspace major.minor – a stale "0.1" would silently resolve to the **previous** release once the workspace moved to 0.2.0, publishing a crate built against its own old sibling.

Tags are per-registry: `rs-v*`, `js-v*`, `py-v*`. One `v0.1.0` firing all three breaks the moment an artifact is unchanged, since crates.io rejects a republished version.

2.4 One compiler for everything we own

The decision this repository fought hardest, worth the detail because the failure mode was invisible.

Our Solidity sources are ranged (`~0.8.20`), so auto-detection had a choice – and made it differently depending on which target ran. `make build` passed `--use 0.8.28`; the bare `forge build` inside `build-uniswap-artifacts` did not, because that one invocation also spanned the Uniswap V2 trigger pinned at `=0.5.16`, and no single `--use` satisfies both. Both wrote `out/`. After a full `make test`, `out/` held **five compiler versions**, with `Buck.json` at 0.8.31 – the very version the pin exists to avoid (an IR codegen bug, "Modifiers not implemented yet").

Which compiler produced a published artifact would have depended on which `make` target ran last.

Now `[profile.default]` declares `solc = "0.8.28"` and `auto_detect_solc = false`. The pin is configuration, not a command-line flag that applies only where someone remembered it. Moving it changes every byte we publish, so it is a version event.

2.5 Third-party artifacts come from their publishers

The Uniswap contracts our worlds deploy are pinned to pre-0.8 pragmas: `v2-core =0.5.16`, `v2-periphery =0.6.6`, `v3-core =0.7.6`. Building them here meant three extra compilers – and foundry on arm64 macOS cannot resolve any of them, failing with "No solc version exists that matches the version requirement" even with the official binaries installed in `~/svm`.

The artifacts that lived in `out/` were **stale leftovers** from a toolchain no longer present here, surviving only because `v2-patch-init-code-hash` guards its rebuild behind `test -f`. The suite's 529 passes were running

against contracts **no current toolchain could reproduce**. A fresh clone, or CI, would simply have failed.

Uniswap publishes those artifacts themselves. `scripts/stage-uniswap.mjs` stages them into `out/` in the forge layout that `vm.deployCode` and both artifact loaders already expect, from exactly-pinned npm packages. This deleted three compilers, two foundry profiles, both compile-trigger directories and the `FOUNDRY_VIA_IR` juggling – and it is exactly what we tell consumers of `alberta-buck-contracts` to do.

Two things this surfaced:

- **The official bytecode is canonical.** `UniswapV2Pair` hashes to `0x96e8ac42...`, the canonical mainnet init-code-hash, so pool addresses now derive exactly as on mainnet and `v2-patch-init-code-hash` has become a no-op that verifies rather than patches.
- **v2's combined-json omits the 0x prefix on bytecode; v3's hardhat output includes it.** Foundry tolerates both, so the forge suite passed either way – but `viem/tevm` reject unprefixed hex, and the JS worlds failed with "deploy WETH9 reverted" until the normalizer forced the prefix. Precisely the class of divergence that two artifact sources conceal.

2.6 The contracts have no addresses

Nothing is deployed at a fixed address, on any chain. Every BUCK world – Python sim, JS sim, browser demos, tests – deploys fresh into its own EVM and learns addresses from the receipts. So `alberta-buck-contracts` ships (`abi`, `bytecode`) pairs, not a deployment registry. `deployments.json` is empty; the slot exists so publishing real addresses later is not a breaking change to the package's shape.

Only the eight contracts we own are published – an allowlist, not a wildcard, because a wildcard over `out/` would sweep in interface stubs, test fixtures and the vendored Uniswap implementations that share the directory. Asking either package for a third-party contract raises an error naming the package to install instead.

`compiler.json` records the exact solc version, optimizer settings, EVM version, git commit and a sha256 over the bundle; the build asserts every artifact was produced by the pinned compiler.

3 Mechanics worth not rediscovering

3.1 The first publish is always manual

Every registry requires a package to **exist** before a trusted publisher can be configured for it:

- **crates.io** – `rust-lang/crates-io-auth-action@v1` needs the crate present; the first publish used a scoped token.
- **npm** – the same, with an open request for OIDC-first-publish (`npm/cli#8544`). Needs `npm >= 11.5.1` and `Node >= 22.14.0`; `ubuntu-latest` ships an older `npm`, so the workflow upgrades it first.
- **PyPI** is the exception: **pending publishers** let a project be created by its first OIDC upload, so PyPI can be zero-token from the start – once the workflow exists and the repository is pushed.

`npm` generates provenance attestations only for **public** repositories. While this repository is private, publishes succeed but carry no provenance, and `--provenance` must not be passed: it fails outright rather than degrading.

3.2 Build outputs are not published by default

`npm` falls back to `.gitignore` when there is no `.npmignore`, and honours it **over** the `files` allowlist. `wasm-pack` writes a `.gitignore` of `"*"` into its `out-dir`, so the first `npm pack --dry-run` of the kernel produced a tarball with a `README`, a `package.json`, and **no `wasm` at all**. The build removes it, and `release-npm.yml` asserts four `.wasm` files in the tarball rather than trusting that it did.

3.3 Dependencies must be pinned to be reproducible

`make install` fetched HEAD of five repositories and `foundry.lock` was git-ignored. OpenZeppelin compiles **into** our contracts, so a 5.6.1 -> 5.7 bump would have changed published bytecode with no commit to show for it. Every dependency now pins an exact tag; `foundry.lock` and `Cargo.lock` are committed.

4 Remaining decisions

4.1 D1: does the sim's source go public?

Publishing a Python package publishes its source: an sdist is a tarball of the package directory, a wheel a zip of .py files. There is no meaningful "binary only" for Python.

The kernels, platform libraries and contract artifacts are published and that is settled. What remains is `alberta_buck` itself – the sim, the equilibrium findings (`EQUILIBRIUM.md`, the sweep vectors, `rebalance_policy.py`), the quote sources.

Recommendation: hold it until the paper it accompanies is out. This is a strategic decision, not a technical one, and the item most likely to be regretted if made by default. It blocks P5 and nothing else.

4.2 D2: where the game's authority lives

Three places the mechanics could be authoritative:

1. **Python server** (`alberta_buck.sim.server`: pyrevm worlds at interactive speed, websockets, multiple clients). Highest fidelity – the real contracts, the real EVM. Requires a host, and `pyrevm` has no linux-aarch64 wheels and none for macOS past cp312, so the target must be x86_64 or build from source.
2. **Browser EVM** (Tevm, per player or per board). Local-first, no server, already exercised by `core/js/test/*.tevm.test.js` – but Tevm is the slow backend and `farmsim.mjs` OOMs past ~6 simulated months.
3. **Kernel-only** – the game computes on `alberta-buck-math` and `alberta-buck-wallet` directly, with no EVM anywhere. The kernels exist precisely because they are bit-identical to the contracts; a game does not need EVM semantics, it needs the arithmetic to be right, and periodic vector checks keep it honest. Runs at native speed via wasm in a browser, and unchanged in a Holochain zome.

Recommendation: **(3), with (1) as the fidelity anchor.** It is the only option that is local-first – the point of choosing Holochain – the only one that runs on a phone, and it makes the wasm kernel rather than a server the game's critical dependency. Under that choice PyPI is the research distribution, not the game's runtime.

This does not make the contract artifacts optional: they build the fidelity anchor, the vector generator compiles them, and any "show me this on a real chain" moment needs them.

5 The way forward

5.1 P2.5b: alberta-buck-contracts on PyPI

The package builds and installs; only a credential is missing. Either configure a PyPI **pending publisher** (project `alberta-buck-contracts`, owner `alberta-buck`, repository `alberta-buck`, workflow `release-pypi.yml`, environment `pypi`) and let the first upload create the project with no token at all – which requires pushing the repository and writing that workflow – or use an API token for the bootstrap, as `npm` and `crates.io` required.

Accept: `pip install alberta-buck-contracts` in a clean venv outside any checkout returns Buck's ABI and 16,158 bytes of bytecode.

5.2 P3: alberta-buck-core on PyPI, and the pure/binary split

`core/python/buck_core/` holds four `.so` files, three of them **byte-identical copies of one dylib** (`buck_identity.so`, `buck_wallet.so`, `buck_registry.so`: 1,491,824 bytes each – one `cdylib` with three `#[pymodule]` entry points). Right for the Makefile, wrong for a wheel: it triples 1.5 MB for nothing and makes `alberta-buck-core` platform-specific, dragging the whole Rust wheel matrix onto the critical path of `pip install alberta-buck`.

Split along the seam the runtime already has:

<code>alberta-buck-core</code>	pure Python, <code>py3-none-any</code> . <code>session</code> , <code>journal</code> , <code>artifacts</code> . Hard dependency of <code>alberta-buck</code> . No Rust in CI.
<code>alberta-buck-kernel</code>	binary, <code>abi3-py311</code> per platform. Optional: <code>'pip install alberta-buck[kernel]'</code> , selected at runtime by <code>BUCK_IDENTITY_BACKEND</code> , as today.

Keep `import buck_core.buck_identity` working with a shim re-exporting from the kernel package when installed, so no calling code changes.

Within the kernel wheel, ship the dylib **once**: Python's extension loader takes the module name and the file separately, and looks up `PyInit_<name>` in whatever file it is given.

```
# buck_kernel/buck_wallet.py -- one .so, three modules
import sys, importlib.util
from importlib.machinery import ExtensionFileLoader
from pathlib import Path

_so = str(Path(__file__).with_name("_kernel.abi3.so"))
_spec = importlib.util.spec_from_file_location(
    __name__, _so, loader=ExtensionFileLoader(__name__, _so))
_mod = importlib.util.module_from_spec(_spec)
_spec.loader.exec_module(_mod)
sys.modules[__name__] = _mod
```

4.9 MB of wheel becomes 1.9 MB. The fallback – ship the dylib three times – is wasteful, not broken.

5.3 P4: alberta-buck-kernel wheels

The only phase with real engineering. `abi3-py311` is already set in both binding crates, which is the large simplification: **one wheel per platform**, not one per Python version, and it keeps working on 3.13, 3.14 and beyond with no rebuild.

Build with **maturin** rather than the plain-cargo Makefile path: it knows `abi3`, emits correct wheel tags, handles the macOS `-undefined dynamic_lookup` flags that `core/rust/.cargo/config.toml` now hand-rolls, and vendors workspace path dependencies into the sdist. Keep `make core-build-py` for development.

Targets: macOS `arm64 + x86_64`, manylinux `_2_28 x86_64 + aarch64`, Windows `amd64`. **Include linux-aarch64** – it is the gap that bites `pyrevm` today, and ARM servers are where a cheap game backend lives.

5.4 P5: alberta-buck on PyPI

Blocked on P3 (hard `buck_core` import) and D1. The contract blocker is cleared: `load_artifact` falls back to the installed `alberta-buck-contracts` when no `foundry.toml` is reachable, with a repo checkout’s `out/` still winning so a rebuild takes effect immediately.

Remaining:

1. **Dependency hygiene.** `matplotlib` (plots only) -> `[plots]`; `websockets` (`sim/server.py` only) -> `[server]`; `pyrevm` (`sim/pyrevm_backend.py` only) -> `[fast]`. `pyrevm` especially: as a hard dependency it makes

`pip install alberta-buck` **fail to build** on linux-aarch64 and on macOS with Python 3.13+, because upstream ships no wheels there. As an extra it is opt-in, and its absence degrades to the anvil backend, which the code already does.

2. Add `alberta-buck-core` and `alberta-buck-contracts` to dependencies, `alberta-buck-kernel` as `[kernel]`, then **delete the `sys.path shim`** in `alberta_buck/__init__.py` – its comment already anticipates this.
3. Align `requires-python` to `>= 3.11` (currently 3.10).
4. Dynamic version from `alberta_buck/version.py`, plus the tag-matches-version CI guard.

5.5 P6: what the game adds

Under D2/(3):

board (projected)		personal device		shared state
-----		-----		-----
<code>alberta-buck-game</code>	<->	<code>alberta-buck-game</code>	<->	Holochain <code>hApp</code>
<code>alberta-buck-core</code>		<code>alberta-buck-core</code>		(zomes: Rust,
<code>alberta-buck-kernel</code>		<code>alberta-buck-kernel</code>		<code>alberta-buck-identity</code> ,
(public: prices, pools, events, narrative)		(private: net worth, holdings, decisions)		<code>alberta-buck-wallet</code> from <code>crates.io</code>)

- `alberta-buck-game` (npm) – rules, turn structure, the narrative price engine (trade wars, embargoes, a mill fire), the mortgage-versus-issuance decision surface. The economics live here; the monetary arithmetic stays in the kernel.
- **Holochain zomes** (Rust, distributed as a `.happ` bundle rather than via a registry), depending on `alberta-buck-identity` and `alberta-buck-wallet` from `crates.io` – the concrete reason the crates went first. The kernels are `std` today; the `no_std` flip is mechanical if a zome needs it.
- **The narrative price generator** is wanted by both the game (JS) and the research sim (Python). If it must agree across both it belongs in the kernel with vectors; if not, keep two simple implementations and say so out loud.

5.6 P7: making the repository public

Independent of everything above. Checklist:

1. Re-run the secrets audit over full history (`.env` is clean and has never been committed; re-verify, and check `foundry.lock`, RPC URLs in committed JSON, and `.claude/`).
2. Decide about the 216 MB of history. Rewriting it – dropping the large circom power tables and PDFs – is a one-time option that disappears the moment the repository is public and forked.
3. `README.org` as a front door; `CONTRIBUTING`; `SECURITY.md` (a monetary system needs a disclosure address); the CLA that dual licensing requires.
4. Branch protection on `master`, requiring the CI gate.
5. Only then do npm provenance attestations begin to be generated.
6. The Solidity *sources* become public here; `alberta-buck-contracts` can then carry a source-verification link.

6 Risks

- **Publishing precedes the papers** (D1). Source on PyPI is source in public. Sequence deliberately.
- **Irreversibility**. crates.io yanks but never deletes; PyPI never re-uses a filename; npm unpublish is 72 hours. Rehearse on TestPyPI, use `--dry-run`, read `cargo package --list` before every first publish.
- **Unaudited contracts in a public registry**. Mitigated by prominent labelling, 0.x versions and an empty `deployments.json` – but someone will eventually deploy them anyway.
- **Binary wheel maintenance** is a standing tax: every new CPython, every new manylinux baseline. abi3 removes the per-version half; the per-platform half remains. Mitigated by `alberta-buck-kernel` being optional by construction.
- **Version skew across registries**. A JS game on kernel 0.2 talking to a Python server on kernel 0.1 is a bit-identity violation no test here

would catch. Mitigated by lockstep at major.minor, and by a cross-registry vector suite each language runs against its *installed* artifact rather than its checkout.

- **Contract/kernel drift.** The kernels are bit-identical to the contracts *at a commit*. On independent version streams nothing enforces the pairing. `compiler.json` records the commit; the kernel crates should record the contract version their vectors came from, and assert it.
- **CAL is unfamiliar.** Counsel will read it rather than recognize it, and some will decline on that alone. Accepted deliberately – the friction is the instrument – but it makes dual licensing, and the CLA, part of the operating cost.
- **Third-party artifact trust.** We ship no Uniswap bytecode, but our worlds execute theirs from npm. Pinned exactly and lockfiled; the init-code-hash check is a standing canary that the bytecode is what mainnet derives from.
- **npm supply chain.** Trusted publishing, the `files` allowlist, and `npm publish --dry-run` in CI. No classic tokens, ever.
- **Holochain maturity** – the D2/(3) recommendation deliberately keeps the game’s monetary correctness independent of it.